

Lab 2A: The Agentic Build Loop

Duration: 30-45 min

After: Installation & Agentic Loop slides

Goal

In traditional development, you would build this application file-by-file, dependency-by-dependency. Today you will describe the system and let an AI engineer build it — then inspect, critique, and extend what it produced.

1. Pre-Flight Check

Open a terminal and verify your environment:

Which terminal? Use whatever you normally use — PowerShell, Windows Terminal, Mac Terminal, or Linux shell. Claude Code works in all of them.

```
node -v
```

Expected: `v18.x.x` or higher

```
git --version
```

Expected: `git version 2.x.x`

```
claude --version
```

Expected: A version number like `1.x.x`

Not installed? See the Installation slide or ask your instructor.

☐ Pre-Flight Complete

- ☐ Node.js 18+ installed
- ☐ Git installed
- ☐ Claude Code installed

2. What You Should See

During the build, Claude will: - Create multiple files automatically - Run `npm install` - Ask permission to run commands - Fix errors without your help

⚠ **Important:** The terminal may pause while Claude reasons or installs packages. **That is normal — do not interrupt it.** When Claude asks for permission, approve it. If you want fewer prompts, select "Yes, and don't ask again."

3. Create Your Project and Start Claude

```
mkdir business-dashboard
cd business-dashboard
claude
```

You're now in an interactive Claude Code session. **From this point forward, everything happens inside Claude.**

4. The Build Prompt

Copy and paste this prompt exactly:

Initialize this as a git repo and build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks – write clean, modern CSS

Database tables:

1. metrics – id, name, value (number), category, recorded_at
2. tasks – id, title, status (pending/in-progress/completed), priority (high/medium/low), assigned_to, created_at, completed_at

API endpoints:

- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority tasks)

Frontend: Dark theme dashboard at / showing metrics as responsive cards and tasks as an interactive list with status badges and priority indicators. Seed the database with 8 sample metrics across categories (financial, customers, operations, hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything, and start the server. Use port 3100 (or PORT env var). Include a .gitignore for node_modules and *.db files.

Watch the agentic loop in action: 1. **Plan** — Analyzes requirements, decides on file structure 2. **Execute** — Creates files, runs `npm install`, writes code 3. **Validate** — Checks for errors, tests the server

Watch how Claude reads terminal errors and fixes them. If Claude hits a dependency failure, a port conflict, or a syntax mistake — it doesn't stop and hand you the error. It reads the error message, reasons about the cause, fixes it, and re-runs automatically. This is the self-healing loop — the difference between a code generator and an autonomous agent.

5. See It in the Browser

Open your browser to **`http://localhost:3100`**

You should see a dark-themed dashboard with metric cards and tasks.

6. Test the System Through Claude

```
Test the API: add a new metric called "New Leads This Week" with value 42
in the "sales" category. Then add a task "Review Q2 projections" with
high priority assigned to me. Then complete task 1.
Show me the results after each operation.
```

Refresh the dashboard in your browser. The new data should appear.

7. Your First Git Commit

```
Commit everything with the message "feat: initial business dashboard with API and UI"
```

***You may see a Skills prompt.** If Claude asks to use a "commit" skill, select **Yes** — this is normal. You'll learn more about Skills in Day 3.*

8. Vague vs. Specific Prompting

Try this vague prompt first:

```
Add validation.
```

Look at what Claude does — it guesses which routes, which fields, what error format.

Now try the specific version:

```
In @src/index.js, modify only the POST /api/tasks route. Validate:
- title: must be non-empty string, max 200 characters
- priority: must be one of "high", "medium", "low"
- assigned_to: optional, but if provided must be non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Compare the outputs. **The specific prompt produces exactly what you want on the first try.**

Specificity isn't extra work — it's less work. One precise prompt beats three vague ones.

9. Quick Reflection

Which part surprised you most?

- ☐ Claude writing the entire project structure
- ☐ Claude fixing its own errors
- ☐ The working UI from a text description
- ☐ The difference between vague and specific prompts

If Something Goes Wrong

Problem	Solution
Claude stops mid-build	Type <code>exit</code> , then <code>claude</code> again. Tell Claude: "The build stopped before finishing. Continue building the dashboard application."
Port 3100 in use	Tell Claude: "Port 3100 is in use. Switch to port 3456."
<code>node -v</code> not found	Install Node.js from nodejs.org (LTS version)
<code>claude</code> not found	Run: <code>curl -fsSL https://claude.ai/install.sh \& bash</code> (Mac/Linux) or see Installation slide (Windows)
Permission errors on npm	Tell Claude: "I'm getting permission errors on npm install. Fix it."
<code>tmpclaude*</code> files appear	Ignore — Claude cleans these up automatically

Go Deeper (Fast Finishers)

Already done with the core lab? Try these progressive challenges:

Reverse Engineer the System

Ask Claude:

```
Show me the full project structure and explain what each file does.
```

Then challenge it:

```
Which parts of this codebase would become technical debt if this were a real product?
```

Controlled Refactor

```
Refactor the backend so all database access is isolated into a db module instead of being mixed with routes. Explain what changed and why this structure is better.
```

Watch Claude restructure the codebase while maintaining functionality.

Add a Feature

```
Add a chart to the dashboard showing metrics grouped by category using Chart.js. Install the dependency and update the frontend.
```

You just added a visual feature to a running system with one prompt.

☐ Lab 2A Checkpoint

- ☐ A running Express server on port 3100
- ☐ SQLite database with metrics and tasks
- ☐ Working web UI at <http://localhost:3100>
- ☐ Input validation on POST `/api/tasks`
- ☐ First git commit

Keep Claude running. Return to slides when your instructor is ready.

© 2026 AIA Copilot — Claude Copilot Training

